

Gherkin & Feature Files

Q1: Explain Scenario Outline vs. Scenario with real-world examples.

Answer:

Scenario is used for fixed input tests. Example:
gherkin

Scenario: Search for a product

Given I am on Amazon

When I search for "laptop"

Then I should see results for "laptop"

Scenario Outline is used for data-driven tests. Example:
gherkin

Scenario Outline: Search for multiple products

Given I am on Amazon

When I search for "<product>"

Then I should see results for "<product>"

Examples:

| product |

| laptop |

| headphones|

| keyboard |

•

Q2: Tests data-driven vs. fixed-input flows.

Answer:

- Fixed-input: One set of hardcoded values per scenario.

- Data-driven: Multiple inputs tested using Scenario Outline and Examples table, improving coverage and reducing duplication.

Q3: How would you design a Scenario Outline to search and add multiple products to an Amazon cart?

Answer:

gherkin

Feature: Add products to cart

Scenario Outline: Search and add product to cart

Given I am logged into Amazon

When I search for "<product>"

And I add "<product>" to the cart

Then "<product>" should be present in the cart

Examples:

| product |

| laptop |

| mouse |

| charger |

Q4: Can a single feature file contain multiple Scenario Outlines? How would you manage their Examples tables?

Answer: Yes, multiple Scenario Outlines are allowed. Each should have its own Examples table to maintain independence and avoid data overlap. Use descriptive scenario names and keep data segregated.

Q5: When would you use Background vs. @Before hooks?

Answer:

- Background: Gherkin-level setup for common steps across scenarios (e.g., login).
- @Before Hook: Code-level setup for initializing browser, DB, or API clients.

Q6: How would you structure a Gherkin scenario for a REST API POST request using Rest Assured?

Answer:

gherkin

Scenario: Create a new user via API

Given the API endpoint "/users"

When I send a POST request with payload:

"""

{

"name": "John",

"email": "john@example.com"

}

"""

Then the response code should be 201

And the response should contain "John"

 Step Definitions & Data Handling

Q7: Demonstrate passing dynamic data from a feature file to step definitions using Cucumber Expressions.

Answer:

gherkin

Scenario: Multiply two numbers

When I multiply {int} and {int}

java

```
@When("I multiply {int} and {int}")  
  
public void multiply(int a, int b) {  
    System.out.println(a * b);  
}
```

Q8: How would you handle complex test data (e.g., nested JSON) in Cucumber?

Answer:

- Use DataTables for structured input.
- Convert to POJOs using Jackson/Gson.
- Load from external JSON files for large payloads.

Q9: Share a code example for reading Excel data into Cucumber tests.

Answer:

java

```
FileInputStream fis = new FileInputStream("data.xlsx");  
  
Workbook wb = new XSSFWorkbook(fis);  
  
Sheet sheet = wb.getSheetAt(0);  
  
String value = sheet.getRow(0).getCell(0).getStringCellValue();
```

Q10: How do you share state between steps across different classes?

Answer:

- Use Context classes to store shared data.
- Apply Dependency Injection via PicoContainer.
- Use ThreadLocal for thread-safe sharing in parallel runs.

 Hooks & Execution Control

Q11: What is the purpose of dryRun=true? When would you use it?

Answer: It validates step-definition mappings without executing tests. Useful during CI setup or refactoring.

Q12: How would you capture screenshots on test failure?

Answer:

```
java
@After
public void tearDown(Scenario scenario) {
    if (scenario.isFailed()) {
        // Capture screenshot logic
    }
}
```

Q13: Implement a tagged hook to run setup only for @Smoke tests.

Answer:

```
java
@Before("@Smoke")
public void smokeSetup() {
    // Setup only for smoke tests
}
```

Q14: How would you skip scenarios dynamically at runtime?

Answer:

Use tag exclusion in runner:
java
tags = "not @Skip"

•

Or conditional logic in step:
java
Assume.assumeTrue(condition);

-

Test Execution & Reporting

Q15: How do you run 50 out of 100 tests using tags?

Answer:

java

tags = "@Smoke or @Regression"

Q16: Explain parallel execution in Cucumber. What challenges might arise?

Answer: Use TestNG/JUnit runners or cucumber-jvm-parallel-plugin. Challenges:

- Thread safety
- Shared state
- Resource contention

Q17: How would you integrate Allure/Extent Reports with Cucumber?

Answer:

java

```
@CucumberOptions(  
    plugin = {"pretty", "io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm"  
})
```

Q18: Configure Cucumber to rerun failed tests automatically.

Answer:

java

```
plugin = {"rerun:target/rerun.txt"}
```

Framework Design

Q19: Design a Cucumber framework supporting both UI (Selenium) and API (Rest Assured) tests.

Answer:

- Use tags to separate UI/API tests.
- Maintain separate step definition packages.
- Share utilities (e.g., config reader, logger).

Q20: How would you structure a large-scale BDD framework?

Answer:

src/test/java

├─ stepdefinitions

├─ runners

├─ hooks

├─ utils

└─ apis

src/test/resources

└─ features

Q21: How do you ensure step definition reusability across features?

Answer:

- Use generic steps (e.g., "I click on {string}").
- Apply DRY principles.
- Create helper classes for common actions.

Q22: What strategies prevent flaky tests in Cucumber?

Answer:

- Use explicit waits.

- Isolate environments.
- Keep scenarios atomic and independent.

Advanced Concepts

Q23: How does Cucumber handle asynchronous operations (e.g., API polling)?

Answer: Implement retry logic with timeouts. Poll until expected condition is met.

Q24: Compare Cucumber Expressions vs. Regex in step definitions.

Answer:

Feature	Cucumber Expressions	Regex
Readability	✅ High	❌ Low
Flexibility	❌ Limited	✅ High
Maintenance	✅ Easier	❌ Complex

Q25: Integrate Cucumber with Jenkins. What reports would you publish?

Answer:

- Use Cucumber CLI or Maven.
- Publish HTML/JSON reports.
- Add stages for build, test, report.

Q26: How would you manage environment-specific configurations?

Answer:

- Use property files per environment.
- Switch via Maven profiles.
- Inject via DI containers.

Troubleshooting & Best Practices

Q27: Debug a scenario where a step definition isn't executed.

Answer:

- Check glue path.
- Ensure step text matches exactly.

Q28: Resolve conflicts when two step definitions match the same Gherkin step.

Answer:

- Ensure unique regex/patterns.
- Use abstract step classes for shared logic.

Q29: Optimize test execution time for 500+ scenarios.

Answer:

- Enable parallelism.
- Use tag filtering.
- Minimize setup in hooks.

Q30: How do you ensure business-readable feature files?

Answer:

- Collaborate with Product Owners.
- Use ubiquitous language.
- Keep steps concise and meaningful.

Missing but Valuable

Q31: How would you version-control feature files with evolving requirements?

Answer:

- Use branching strategies (e.g., GitFlow).
- Apply Gherkin linting for consistency.

Q32: Implement custom Cucumber plugin for audit logging.

Answer:

- Implement Cucumber plugin interface.

Gherkin Advanced Usage

Q33: How would you handle localization/multi-language feature files?

Answer:

gherkin

features/es/search.feature

Característica: Búsqueda

Escenario: Búsqueda básica

Dado que estoy en la página principal

Use cucumber --i18n es to run Spanish features. Maintain separate feature files per language with identical step definition mappings.

Q34: Explain the @ symbol usage in Scenario Outlines with examples

Answer:

gherkin

Scenario Outline: Payment with <card>

Given payment method @<card>

Examples:

card	
visa	# Becomes "@visa"
mastercard	

The @ injects values directly into Gherkin steps, useful for dynamic selectors or API endpoints.

Step Definition Techniques

Q35: How would you implement custom parameter types for domain objects?

Answer:

java

@ParameterType(".*? product")

```
public Product product(String name) {  
    return new Product(name.replace(" product", ""));  
}
```

// Usage

@When("I add {product} to cart")

```
public void addProduct(Product product) {  
    // ...  
}
```

Registers converters for custom syntax like "I add 'laptop product' to cart".

Q36: Handle dynamic data tables with header mismatches

Answer:

gherkin

Then I see:

FIELD	VALUE
Name	John

```
| Email | john@ex |
java
@Then("I see:")
public void verify_data(Map<String, String> expected) {
    // Auto-converts to map ignoring header row
    assertThat(actualData).containsAllEntriesOf(expected);
}
```

Cucumber automatically converts tables with headers to key-value maps.

Hooks & Execution

Q37: Implement conditional hook execution based on tags

Answer:

```
java
@Before(value = "@UI", order = 10)
public void initBrowser() {
    // Runs only for @UI tagged scenarios
}
@After("@API")
public void teardownAPI() {
    // API cleanup
}
```

Use tag expressions in hooks for environment-specific setup/teardown.

Q38: How would you share state between hooks and step definitions without DI containers?

Answer:

```
java
public class TestContext {
    public static ThreadLocal<Map<String, Object>> data =
        ThreadLocal.withInitial(HashMap::new);
}
@Before
public void setup(Scenario scenario) {
    TestContext.data.get().put("scenario", scenario);
}
@When("I do action")
public void action() {
    Scenario sc = (Scenario) TestContext.data.get().get("scenario");
}
```

ThreadLocal provides thread-safe state sharing for parallel execution.

Data Management

Q39: Implement lazy loading of external test data

Answer:

```
gherkin
Scenario Outline: Login
    When I login with <credential>
```

Examples:

```

| credential |
| ${data.file:credentials.json#valid} |
| ${data.file:credentials.json#invalid} |
java
@DefaultParameterTransformer
public Object transform(String fromValue, Type toValueType) {
    if (fromValue.startsWith("${data.file}")) {
        return loadFromExternalFile(fromValue); // Custom loader
    }
    return defaultTransformer.transform(fromValue, toValueType);
}

```

Extend Cucumber's parameter transformer to handle custom placeholders.

Integration Patterns

Q40: How would you integrate Cucumber with Kafka for event validation?

Answer:

gherkin

Scenario: Order event

When I place order #123

Then I should see "OrderCreated" event on Kafka topic "orders"

```

java
@Then("I should see {string} event on Kafka topic {string}")
public void verifyKafkaEvent(String eventType, String topic) {
    KafkaConsumer consumer = new KafkaConsumer(topic);
    assertThat(consumer.getEvents())
        .filteredOn(e -> e.type.equals(eventType))
        .isNotEmpty();
}

```

Use dedicated steps for message queue assertions with domain-specific language.

Testing Best Practices

Q41: How would you validate accessibility compliance in Cucumber?

Answer:

gherkin

Scenario: Homepage accessibility

Given I'm on homepage

Then it should pass WCAG 2.1 AA standards

```

java
@Then("it should pass {string} standards")
public void verifyAccessibility(String standard) {
    AxeBuilder builder = new AxeBuilder()
        .withTags(standard.split(" "));
    assertThat(builder.analyze(driver)).hasNoViolations();
}

```

Encapsulate accessibility checks in reusable steps with tools like Axe-core.

Meta-Programming

Q42: How would you dynamically generate scenarios from external data?

Answer:

```

java
public class DynamicScenarios implements ScenarioGenerator {
    @Override
    public void generate(FeatureBuilder featureBuilder) {
        for (Product p : ProductLoader.load()) {
            featureBuilder.scenario("Test " + p.name())
                .given("I search for '" + p.name() + "'")
                .then("I see results");
        }
    }
}

```

Implement ScenarioGenerator for programmatic scenario creation (requires custom runner).

ADVANCE QUESTIONS:

🕒 Step Definition Techniques & Advanced Gherkin

Q31: Explain the use of the @ symbol in Scenario Outlines for dynamic value substitution.
Answer:

The @ symbol is used directly within the Gherkin steps of a Scenario Outline to inject values from the Examples table. This is useful for making steps read more naturally or for matching dynamic selectors.

gherkin

Scenario Outline: Filter products by brand

Given I am on the product search page

When I filter by the brand "@<brand_name>"

Then I should only see products from "@<brand_name>"

Examples:

brand_name
Apple
Samsung

In this case, the step becomes "I filter by the brand '@Apple'", which might be exactly what a dropdown option or API parameter expects.

Q32: How would you implement a custom parameter type (e.g., to convert a string into a Product object)?

Answer:

Custom parameter types allow you to define your own transformation logic from a Gherkin expression to a complex domain object, making steps more expressive and type-safe.

java

// Define the custom parameter type (usually in a hook or configuration class)

```
@ParameterType(".*") // Matches any string
public Product product(String name) {
    return new Product(name); // Convert the string into a Product object
}
```

// Use it in a step definition

```
@When("I add the {product} to the cart")
public void i_add_the_product_to_the_cart(Product product) {
    // 'product' is now a Product object, not a String
    cart.add(product);
}
```

// The corresponding Gherkin step

When I add the "MacBook Pro" to the cart

Q33: How do you handle a DataTable where the header row does not match the expected keys exactly?

Answer:

You can process the DataTable as a list of maps and manually handle the key mismatches, or you can use a transformer for more complex logic.

java

```
@Then("the following user details should be correct:")
public void the_following_user_details_should_be_correct(io.cucumber.datatable.DataTable dataTable) {
    // 1. Convert DataTable to a List<Map<String, String>>
    List<Map<String, String>> userDetailsList = dataTable.asMaps(String.class, String.class);
    Map<String, String> expectedDetails = userDetailsList.get(0); // Get first row

    // 2. Handle key aliases or mismatches manually
    String name = expectedDetails.get("Full Name"); // if header is "Full Name"
    String email = expectedDetails.getOrDefault("Email Address", expectedDetails.get("E-mail")); // handle multiple possible keys

    // ... perform assertions
}
```

⚙ Hooks & Execution Control (Advanced)

Q34: How would you implement a conditional hook that only runs for a specific combination of tags (e.g., @UI and not @Mobile)?

Answer:

Cucumber supports tag expressions, which allow for boolean logic in hook conditions.

```
java
```

```
// This hook will run before every scenario tagged with @UI but NOT tagged with @Mobile
```

```
@Before("@UI and not @Mobile")
public void setupDesktopBrowser() {
    // Initialize a desktop browser
}
```

```
// This hook will run before scenarios tagged with either @Smoke OR @Regression
```

```
@Before("@Smoke or @Regression")
public void setupForCriticalTests() {
    // Special setup for critical test paths
}
```

Q35: How do you share state between steps and hooks without using a DI container like PicoContainer?

Answer:

A common pattern is to use a static **ThreadLocal** holder class to ensure thread-safety, especially in parallel test execution.

```
java
```

```
public class TestContext {
    // ThreadLocal ensures each thread has its own copy of the context map
    private static final ThreadLocal<Map<String, Object>> CONTEXT =
    ThreadLocal.withInitial(HashMap::new);
```

```
    public static void set(String key, Object value) {
        CONTEXT.get().put(key, value);
    }
```

```
    public static <T> T get(String key, Class<T> type) {
        return type.cast(CONTEXT.get().get(key));
    }
```

```
    public static void cleanup() {
        CONTEXT.remove(); // Crucial to prevent memory leaks
    }
}
```

```
// In a hook
```

```
@Before
public void setUp(Scenario scenario) {
    TestContext.set("currentScenario", scenario);
}
```

```
// In a step definition
```

```
@When("I do something")
public void i_do_something() {
    Scenario scenario = TestContext.get("currentScenario", Scenario.class);
    scenario.log("Performing an important step...");
}
```

// In an after hook

```
@After
public void tearDown() {
    TestContext.cleanup();
}
```

Data Management & External Files

Q36: How would you lazy-load test data from an external JSON or YAML file for your Examples table?

Answer:

Cucumber doesn't support this natively, but you can use a custom method to read the external file and feed it into a **Scenario Outline** dynamically.

1. Create a utility to load data:

java

```
public class DataLoader {
    public static Object[][] loadTestDataFromJson(String filePath) {
        // Use Jackson, Gson, etc. to read JSON and convert to a 2D Object array
        // Example: [ ["laptop"], ["headphones"] ]
        return loadedData;
    }
}
```

2. Use a dedicated runner or a **@Before** hook to inject data (this is a conceptual approach; direct injection isn't standard). A more common pattern is to use a **@DataProvider** with TestNG or the **@ParameterizedTest** with JUnit 5, but for pure Cucumber, you might generate a temporary feature file.

A simpler, more pragmatic solution is to use the **Examples** path:

gherkin

Scenario Outline: Login with external data

When I login as <username> with password <password>
Then I should see the <outcome>

Examples: {file:test-data/login.json}

This requires a custom Cucumber plugin or extension, which is an advanced implementation.

Q37: How would you integrate Cucumber with a message queue like Kafka to validate emitted events?

Answer:

Create step definitions that interact with a Kafka consumer to listen for and validate messages.

java

```
public class KafkaSteps {

    private KafkaConsumer<String, String> consumer;

    @Before("@Kafka")
    public void setupKafka() {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "cucumber-test");
        consumer = new KafkaConsumer<>(props);
    }

    @Then("a {string} event for order {string} is published to the {string} topic")
    public void verifyEventPublished(String eventType, String orderId, String topic) {
        consumer.subscribe(Collections.singletonList(topic));
        ConsumerRecords<String, String> records =
        consumer.poll(Duration.ofSeconds(10));

        boolean found = false;
        for (ConsumerRecord<String, String> record : records) {
            if (record.value().contains(eventType) && record.value().contains(orderId)) {
                found = true;
                break;
            }
        }
        assertTrue("Expected event was not found", found);
    }

    @After("@Kafka")
    public void tearDownKafka() {
        consumer.close();
    }
}
```

 Framework Design & Best Practices

Q38: How would you validate accessibility (a11y) compliance within your Cucumber UI tests?

Answer:

Integrate an accessibility engine like Axe-core with Selenium and create a reusable step.

java

```
@Then("the page should have no accessibility violations")
public void the_page_should_have_no_accessibility_violations() {
    AxeBuilder axeBuilder = new AxeBuilder();
    // Configure AxeBuilder if needed (e.g., with tags)
    axeBuilder.withTags(Arrays.asList("wcag2a", "wcag2aa"));

    Results results = axeBuilder.analyze(driver);

    if (results.getViolations().isEmpty()) {
        return; // Test passes
    } else {
        String violationMessage = new
        AxeReportBuilder().buildReport(results.getViolations());
        throw new AccessibilityException("Accessibility violations found: " +
        violationMessage);
    }
}
// Gherkin
Then the page should have no accessibility violations
```

Q39: What strategies do you use to keep your feature files clean and maintainable when dealing with complex user workflows?

Answer:

- Abstract Common Flows: Use higher-level steps that encapsulate complex sequences (e.g., Given I have a checked-out cart with a "laptop" instead of all the steps to search, add, and checkout).
- Use Background Effectively: Put common pre-conditions for all scenarios in the feature file in the Background section.
- Leverage Tags: Use tags for filtering and to provide metadata about features/scenarios (@slow, @checkout_flow).
- Rule Keyword: Use the Rule keyword to group related scenarios and provide context, improving narrative structure.
- Break Down Large Features: Don't be afraid to split one giant checkout.feature into cart_management.feature, shipping_selection.feature, and payment_processing.feature.

Q40: How would you dynamically generate test scenarios at runtime based on external data?

Answer:

This is an advanced topic. Pure Cucumber is designed for static feature files. To achieve dynamic generation, you would typically:

1. Pre-process: Write a script that generates a `.feature` file from your external data source before the Cucumber execution starts.
2. Use JUnit/TestNG Parameterization: Forgo the Gherkin layer for highly dynamic data and use the underlying test framework's parameterization, while still using Cucumber's glue code for steps.
3. Custom Runner/Plugin: Create a custom Cucumber plugin that implements `ScenarioGenerator` to create scenarios programmatically. This is complex and rarely needed for most projects.